Jupyter Notebook Demonstration: Time Series Visualization

In the following video demonstration, you will see a walk-through and detailed explanation of the provided Example Jupyter Notebook. This Notebook loads a database of daily minimum temperature and then generates graphs using Matplotlib library.

Here, you will step through the example code and explain each block's function.

Part 1: Control Chart

Code:

```
import matplotlib.pyplot as plt
import numpy as np
```

Explanation: The above lines import the ***matplotlib*** and ***numpy*** libraries for plotting the graph and handling data respectively.

Code:

```
data =np.genfromtxt('daily-minimum-temperatures-in-me.csv',
        delimiter=',',skip_header=1)
temps = data[:,1]
```

Explanation: The first line of code reads the data from the 'Daily minimum temperatures in Melbourne' file using the ***genfromtxt*** function of numpy. The second line, we select only the temperature data.

Code:

```
mean = np.nanmean(temps)
std = np.nanstd(temps)
```

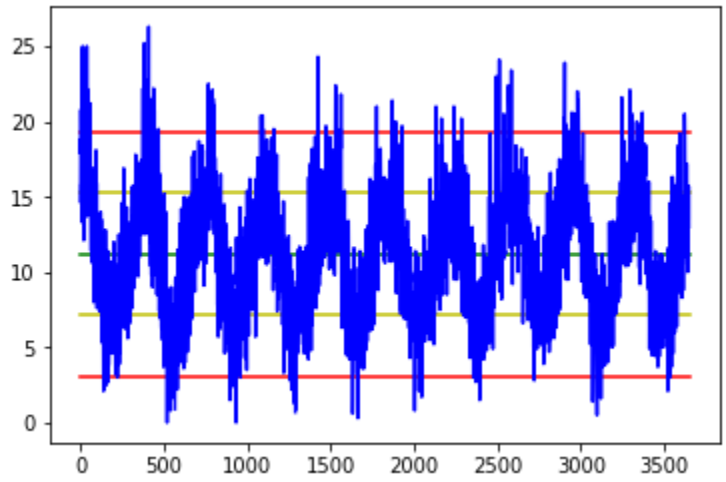
Explanation: In the above two lines, we compute the mean and standard deviation of the temperature values. We are using the nanmean and nanstd functions in numpy to handle not a number or NaN elements in the array.

Code:

```
plt.plot([0,len(temps)], [mean, mean], 'g-')
plt.plot([0,len(temps)], [mean+std, mean+std], 'y-')
plt.plot([0,len(temps)], [mean-std, mean-std], 'y-')
plt.plot([0,len(temps)], [mean+2*std, mean+2*std], 'r-')
plt.plot([0,len(temps)], [mean-2*std, mean-2*std], 'r-')
plt.plot(range(len(temps)), temps, 'b-')
plt.show()
```

Explanation: In the above code, we plot the mean, mean plus standard deviation, mean plus two times standard deviation, mean minus standard deviation and mean minus two times standard deviation as lines of constant value. Then we plot the actual temperatures. In the third arguments to each plot function, we are explicitly telling matplotlib to give the lines a specific color and line style for better understanding. In this case, ‘g-’ instructs matplotlib to plot the mean line as a green straight solid line, ‘y-’ instructs matplotlib to plot the mean plus standard deviation and mean minus standard deviation as a yellow solid line. The ‘r-’ does the same thing for mean plus and minus two standard deviations as a red straight line. Finally, ‘b-’ instructs the matplotlib to plot the actual temperature data as a blue straight line with no decorations on the individual point values.

Resultant Plot:



Part 2: Moving Average Plot

Code:

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

Explanation: The above lines import the ***matplotlib*** and ***numpy*** libraries for plotting the graph and handling data respectively.

Code:

```
data =np.genfromtxt('daily-minimum-temperatures-in-me.csv',
        delimiter=',',skip_header=1)
temps= data[:,1]
```

Explanation: The above lines reads the dataset file and selects only temperature values same as in Part 1.

Code:

```
window_size = 50
```

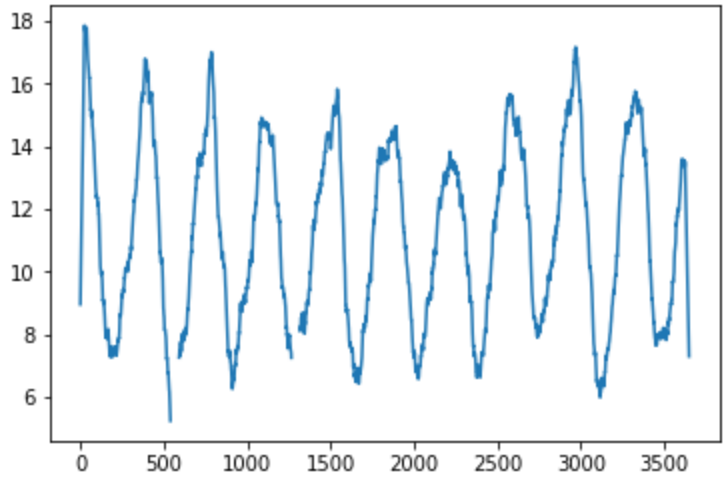
Explanation: The above line of code describes the size of rolling window over to average. Our dataset represents daily temperatures, we have chosen window size to 50 as it corresponds to approximately a month and a half of data averaged together.This allows us to average out individual daily shifts in temperature,but doesn't average out seasonal swings in temperature.

Code:

```
plt.plot(np.convolve(temps, np.ones(window_size,)/window_size, 'same'))
plt.show()
```

Explanation: In the first line of code, we compute the moving average using the convolve function of Numpy and plot the chart.

Resultant Plot:



Part 3: Exponential Weighted Moving Average Plot

Code:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

Explanation: The first two lines, we import the ***matplotlib*** and ***numpy*** libraries for plotting the graph and handling data respectively. In the last line, we import the ***pandas*** library which has the built in exponential weighted moving function

Code:

```
data =np.genfromtxt('daily-minimum-temperatures-in-me.csv',
        delimiter=',',skip_header=1)
temps= data[:,1]
```

Explanation: The above lines reads the dataset file and selects only temperature values same as in Part 1 and 2.

Code:

```
span =50
df = pd.DataFrame({'temps':temps})
```

Explanation: In the first line, span serves the same purpose as 'window size' in the weighted moving average plot. In the second line, we convert the numpy array of temperatures into a pandas dataframe so that we can use the built-in functions.

Code:

```
df_exp = df.ewm(span=50).mean()
plt.plot(df_exp)
plt.show()
```

Explanation: In the first line, we use the ewm function of the dataframe which provides an exponential weighting window as well as the mean function to compute the exponential weighted moving average of temperature values. The last two lines plot the graph.

Resultant plot:

